

CO5_ASSESSMENT 1

CONSTRAINT -BASED PROBLEM SOLVING

1. Real-Time Face Recognition for Campus Security

A university wants to deploy a camera-based security system at the entrance that can identify registered students and staff in real time.

The system should:

- Capture faces from live CCTV/video streams.
- Detect faces under different lighting conditions.
- Extract suitable facial features.
- Recognize registered individuals.
- Maintain real-time performance on a low-cost computer.
- Minimize false recognition.

Design a complete OpenCV-based face recognition pipeline from image acquisition to final identification. Explain the preprocessing, face detection, feature extraction, recognition, and optimization techniques used. Justify how your system achieves accuracy and real-time performance.

2. Intelligent Traffic Violation Detection System

A traffic department wants to automatically monitor vehicles using roadside cameras.

The system should:

- Detect vehicles from live video.
- Track vehicles across consecutive frames.
- Identify vehicles crossing a predefined line.
- Count vehicles accurately.
- Detect abnormal or unwanted movements.
- Display vehicle count and alerts in real time.

Design an OpenCV-based solution incorporating video acquisition, preprocessing, object detection, motion analysis, tracking, and visualization. Explain how you would handle shadows, camera noise, dynamic backgrounds, and multiple vehicles to minimize counting errors.

3. Automated Document Processing System

A company receives thousands of scanned documents that may contain blurred text, uneven illumination, noise, and different font sizes.

The system should:

- Automatically improve document quality.
- Remove noise and unwanted background.
- Convert the document into a suitable format for OCR.
- Segment text regions.
- Extract readable text efficiently.
- Process a large number of documents in batch mode.

Design an OpenCV-based image processing pipeline for this system. Explain the role of grayscale conversion, thresholding, filtering, morphological operations, segmentation, and geometric correction. Justify how each stage improves OCR accuracy.

4. Real-Time Multi-Person Tracking and Attendance System

A college wants to automatically record student attendance using classroom CCTV cameras.

The system should:

- Detect multiple faces simultaneously.
- Track each person across video frames.
- Recognize registered students.
- Avoid repeatedly marking the same student.
- Handle slight head movement and temporary face disappearance.
- Operate continuously with minimum processing delay.

Design a complete OpenCV-based system using face detection, feature extraction, recognition, and tracking. Explain how you maintain identity across frames and prevent false or duplicate attendance records.

5. Smart Home Surveillance and Intrusion Detection

A home security system uses a fixed camera to monitor an entrance continuously.

The system should:

- Detect significant motion.
- Distinguish actual movement from camera noise and small background changes.
- Track moving objects.
- Identify suspicious activity.
- Generate an alert when an unauthorized person enters a predefined region.
- Operate in real time with limited computational resources.

Design an OpenCV-based surveillance pipeline using background subtraction, filtering, morphological operations, contour/feature detection, motion analysis, and tracking. Explain how your approach reduces false alarms caused by shadows, illumination changes, trees, and other dynamic background objects.

CO5 – ASSESSMENT TOOL 1

CONSTRAINT-BASED PROBLEM SOLVING

OPEN-CV BASED COMPUTER VISION SOLUTIONS

Student Name: K Nitheesh Kumar

Register Number: 192325090

Assignment Coverage

The supplied assessment contains five OpenCV-based problem statements: real-time face recognition for campus security, intelligent traffic violation detection, automated document processing, real-time multi-person tracking and attendance, and smart home surveillance/intrusion detection. This document provides detailed answers for all five scenarios so that the selected question can be used directly if the faculty assigns one of them. The source specifically asks for complete pipelines, preprocessing, detection/feature extraction or segmentation, optimization, and justification of accuracy and real-time performance. 穀filecite學turn14file0學L2-L17像

Student Details

Field	Value
Student Name	K Nitheesh Kumar
Register Number	192325090
Course / Assessment	CO5 Assessment 1 – Constraint-Based Problem Solving
Technology Focus	OpenCV and Computer Vision
Signature	K Nitheesh Kumar

QUESTION 1 – REAL-TIME FACE RECOGNITION FOR CAMPUS SECURITY

The first scenario asks for a camera-based system at a university entrance that identifies registered students and staff in real time. The source requirements include live CCTV/video capture, face detection under different lighting conditions, facial feature extraction, recognition of registered individuals, real-time performance on a low-cost computer and minimization of false recognition. 穀filecite學turn14file0學L4-L11條

1.1 Proposed Pipeline

```
Live CCTV / Video
↓
Frame Acquisition
↓
Resize + Color Conversion
↓
Illumination / Noise Preprocessing
↓
Face Detection
↓
Face Alignment + Normalization
↓
Feature Extraction
↓
Feature Matching / Classifier
↓
Confidence Threshold
↓
Identity / Unknown
↓
Logging + Display
```

1.2 Image Acquisition

Frames are captured continuously from a CCTV stream using OpenCV VideoCapture. Processing every full-resolution frame is unnecessary on a low-cost computer, so the system can sample frames or resize them to a manageable resolution while preserving sufficient facial detail. A frame timestamp and camera identifier should be retained for traceability.

1.3 Preprocessing

- Convert BGR frames to grayscale when the selected detector operates on intensity information.
- Resize frames to reduce computational cost while maintaining usable face resolution.
- Apply mild denoising when camera noise is significant.
- Normalize illumination where strong brightness variation affects recognition.
- Use face alignment after detection to reduce variation caused by head position.
- Reject extremely small or blurred face crops that are unlikely to produce reliable features.

1.4 Face Detection

A practical OpenCV implementation can use a Haar cascade or a more robust DNN-based detector. The detector produces bounding boxes around faces. For varying lighting and pose, a trained DNN detector is generally preferable when hardware permits, while a Haar cascade is lighter for low-cost devices. Detection should be followed by size and confidence filtering.

QUESTION 1 – PIPELINE DETAILS AND OPTIMIZATION

1.5 Face Alignment

After a face is detected, the crop is normalized to a common size. If facial landmarks are available, the eyes and other landmarks can be used to align the face. Alignment reduces variation caused by rotation and improves consistency of feature vectors.

1.6 Feature Extraction

The system converts the aligned face into a numerical representation. Depending on the implementation, this may be a deep face embedding model or a classical feature representation. Deep embeddings are preferable for identity recognition because similar images of the same person tend to map closer together in feature space.

1.7 Recognition

Registered students and staff are enrolled in advance. For each registered person, one or more feature vectors are stored. A live face embedding is compared with stored vectors using a distance measure such as Euclidean or cosine distance. The closest candidate is accepted only when the similarity exceeds a chosen confidence threshold; otherwise the system reports 'Unknown'.

Stage	Technique	Purpose
Acquisition	VideoCapture	Obtain live frames.
Preprocessing	Resize, grayscale/normalization	Reduce noise and computation.
Detection	Haar/DNN face detector	Locate faces.
Alignment	Landmark-based crop	Normalize pose.
Feature extraction	Face embedding	Represent identity.
Recognition	Distance/classifier	Match to registered users.
Decision	Thresholding	Reduce false recognition.
Output	Overlay/log	Show and record result.

1.8 False Recognition Reduction

- Use a conservative recognition threshold rather than accepting every nearest neighbor.
- Require the same identity to be recognized across multiple consecutive frames.
- Maintain an Unknown class for low-confidence matches.
- Use multiple enrollment images per person under different conditions.
- Reject low-resolution and severely blurred face crops.
- Periodically evaluate false acceptance and false rejection on a validation dataset.

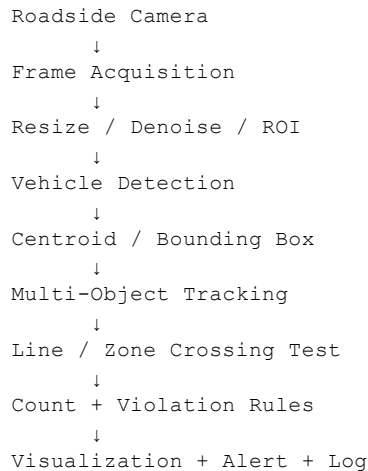
1.9 Real-Time Optimization

- Resize the input frame before detection.
- Process every Nth frame while tracking detected faces between recognition frames.
- Use a lightweight detector on constrained hardware.
- Run recognition only on detected face regions rather than the entire frame.
- Use vectorized numerical operations and avoid unnecessary image copies.
- Separate capture, inference and display into efficient processing stages.

QUESTION 2 – INTELLIGENT TRAFFIC VIOLATION DETECTION

The second scenario requires live roadside cameras to detect vehicles, track them across frames, identify vehicles crossing a predefined line, count vehicles, detect abnormal or unwanted movements and display counts and alerts in real time. The source explicitly asks for video acquisition, preprocessing, object detection, motion analysis and visualization, including handling shadows, camera noise, dynamic backgrounds and multiple vehicles. 數filecite學turn14file0學L12-L17倅

2.1 Proposed Pipeline



2.2 Region of Interest

A fixed roadside camera often contains irrelevant regions such as sky, buildings and sidewalks. Defining a region of interest reduces the number of pixels processed and prevents unrelated objects from triggering vehicle detections. The ROI should include the lanes and the virtual counting line.

2.3 Object Detection

Vehicle detection can be implemented using background subtraction for a fixed camera or a trained object detector when reliable classification of cars, buses, trucks and motorcycles is required. OpenCV can perform preprocessing and post-processing around the detector. Bounding boxes are filtered by minimum size and plausible road position.

2.4 Motion Analysis

For a fixed camera, background subtraction can identify moving foreground objects. Gaussian Mixture-based background modelling can adapt to gradual changes. However, raw foreground masks contain noise and shadows, so morphological processing and contour filtering are required.

QUESTION 2 – TRACKING, COUNTING AND VIOLATIONS

2.5 Morphological Processing

- Opening removes small isolated noise.
- Closing fills small gaps in vehicle silhouettes.
- Erosion can reduce thin unwanted regions.
- Dilation can reconnect fragmented foreground regions.
- Contour-area filtering removes tiny objects.

2.6 Multi-Vehicle Tracking

Each detected vehicle is assigned a track identifier. A simple centroid tracker can match detections by distance between current and previous centroids. A stronger tracker can combine appearance, motion and bounding-box overlap. The tracker maintains the identity of each vehicle across consecutive frames and tolerates brief detection failures.

2.7 Virtual Line Crossing

A predefined line is placed across the lane. For each tracked vehicle, the system stores the previous centroid and current centroid. If the centroid changes from one side of the line to the other, the vehicle is counted once. The track stores a 'counted' flag so that the same vehicle is not counted repeatedly.

```
if previous_y < line_y and current_y >= line_y:
    if not track.counted:
        vehicle_count += 1
        track.counted = True
        create_alert_if_required()
```

2.8 Handling Shadows and Dynamic Backgrounds

Problem	Mitigation
Shadows	Use shadow detection where supported; compare shape/brightness features.
Camera noise	Apply blur before background subtraction.
Trees / moving background	Use adaptive background modelling and ROI restrictions.
Lighting change	Update background gradually and use robust thresholds.
Multiple vehicles	Use object IDs and per-track state.
Occlusion	Use tracker prediction and re-identification where available.

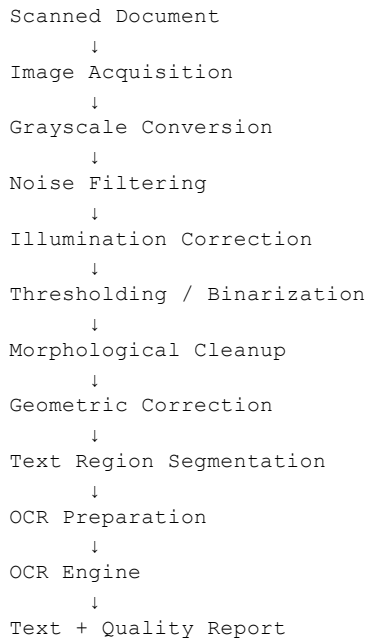
2.9 Real-Time Visualization

The display should overlay bounding boxes, object IDs, the virtual counting line, current vehicle count, violation labels and timestamp. Alerts can be generated when a vehicle crosses a restricted line or exhibits a rule-defined abnormal movement.

QUESTION 3 – AUTOMATED DOCUMENT PROCESSING

The third scenario describes thousands of scanned documents with blurred text, uneven illumination, noise and different font sizes. The system must improve document quality, remove noise/background, prepare images for OCR, segment text regions, extract readable text and support batch processing. The source specifically asks for grayscale conversion, thresholding, filtering, morphology, segmentation and geometric correction. 穀filecite孳turn14file0孳L18-L25倅

3.1 Proposed Image Processing Pipeline



3.2 Grayscale Conversion

OCR usually does not need the full color information of a document. Converting a color scan to grayscale reduces the image to intensity information and simplifies subsequent thresholding and filtering. It also reduces computational and memory requirements.

3.3 Filtering

A Gaussian or median filter can reduce high-frequency noise. Median filtering is particularly useful for impulse-like speckle noise while preserving edges better than a simple averaging filter. The filter strength should be controlled because excessive smoothing can remove thin character strokes.

3.4 Uneven Illumination Correction

Uneven lighting causes one part of the document to appear brighter than another. Adaptive thresholding can determine local thresholds instead of applying one global threshold to the whole page. Background estimation and normalization can also reduce large-scale illumination variation.

QUESTION 3 – THRESHOLDING, MORPHOLOGY AND OCR

3.5 Thresholding

Thresholding converts a grayscale image into a binary representation separating probable foreground text from background. Global thresholding is effective for uniformly illuminated documents, while adaptive thresholding is more appropriate when illumination varies across the page. Otsu's method can automatically select a global threshold when the intensity histogram is suitable.

3.6 Morphological Operations

Operation	Effect on Document
Opening	Removes small noise and isolated dots.
Closing	Connects broken character/line regions.
Dilation	Thickens foreground and can reconnect strokes.
Erosion	Thins foreground and removes small protrusions.

3.7 Geometric Correction

A scanned document may be rotated or skewed. Deskewing aligns text lines horizontally, making segmentation and OCR easier. Perspective correction can be used when the page is photographed at an angle rather than scanned flat.

3.8 Text Segmentation

The cleaned document can be segmented into lines, paragraphs or blocks using contours, projection profiles or connected components. Segmentation reduces the amount of irrelevant background presented to OCR and can help the OCR engine determine the expected layout.

3.9 Batch Processing

- Process documents independently so one failed file does not stop the batch.
- Use a queue to distribute files among worker processes.
- Store original and processed versions with unique identifiers.
- Record processing status, errors and OCR confidence.
- Reuse loaded models/configuration rather than reinitializing for every page.
- Compress intermediate representations when storage is constrained.

3.10 Accuracy Justification

Each preprocessing stage addresses a known OCR difficulty. Grayscale simplifies representation, filtering reduces noise, illumination correction improves foreground/background separation, thresholding creates a clear text mask, morphology repairs or removes small structures, and geometric correction improves text orientation. The result is a cleaner input for OCR.

QUESTION 4 – REAL-TIME MULTI-PERSON TRACKING AND ATTENDANCE

The fourth scenario requires simultaneous face detection, tracking, recognition, duplicate-attendance prevention, tolerance of slight head movement and temporary face disappearance, and continuous operation with minimum processing delay. The source asks for a complete OpenCV system using face detection, feature extraction, recognition and tracking.

4.1 Proposed Pipeline

```
Classroom CCTV
↓
Frame Capture
↓
Person/Face Detection
↓
Face Crop + Alignment
↓
Feature Extraction
↓
Identity Matching
↓
Multi-Object Tracking
↓
Track Persistence
↓
Attendance State Check
↓
Mark Once + Timestamp
↓
Attendance Database / Report
```

4.2 Detection and Recognition

The system first detects multiple faces in each selected frame. Each face is aligned and transformed into an embedding. The embedding is compared against enrolled student embeddings. A confidence threshold prevents uncertain matches from being automatically marked.

4.3 Identity Persistence

Recognition does not need to run on every frame. Once a face is recognized, a tracker can maintain the location between recognition intervals. Each track stores an ID, bounding box, last-seen timestamp, identity hypothesis and recognition confidence.

4.4 Temporary Disappearance

A person may be hidden briefly by another student or turn away from the camera. The track should therefore survive a configurable number of missing detections. If the face reappears near the predicted location, the tracker can reconnect the observation to the existing track.

4.5 Preventing Duplicate Attendance

Attendance should be stored by student ID and class/session ID with a unique constraint. Once a student's attendance state changes from 'not marked' to 'marked', subsequent frames must not create another record.

```
if recognized_id and confidence >= threshold:
    if not attendance[session_id].contains(recognized_id):
        mark_attendance(recognized_id, timestamp)
        attendance[session_id].add(recognized_id)
    else:
        update_last_seen(recognized_id)
```

QUESTION 4 – TRACKING QUALITY AND REAL-TIME PERFORMANCE

4.6 Handling Head Movement

- Use an embedding model tolerant of moderate pose variation.
- Use face alignment before feature extraction.
- Maintain track identity even when the face is briefly less frontal.
- Require multiple consistent recognition observations before final marking.
- Use a temporal confidence score rather than a single-frame decision.

4.7 False Attendance Prevention

Risk	Control
Wrong identity	Conservative similarity threshold.
One-frame false match	Require repeated consistent observations.
Duplicate record	Unique student/session key.
Temporary occlusion	Track persistence.
Poor-quality face	Reject small/blurred crops.
Two people close together	Use multi-object tracking and separate boxes.

4.8 Optimization

- Run face detection at a reduced frequency.
- Track between detection frames.
- Resize the classroom video while retaining adequate face size.
- Batch-process face crops when the model supports it.
- Use lightweight models when hardware is limited.
- Keep attendance database operations asynchronous where possible.

4.9 Attendance State Machine

```
UNSEEN
  ↓ recognized with sufficient confidence
CANDIDATE
  ↓ consistent observations
PRESENT
  ↓
ATTENDANCE MARKED
  ↓
Continue tracking / last-seen updates
```

```
If confidence falls temporarily:
PRESENT → TRACKING
If absent beyond timeout:
TRACKING → UNSEEN
```

4.10 Output

The classroom dashboard can display current recognized students, number present, current tracks, confidence state and last-seen time. The final attendance record should contain student ID, session ID, first recognition time and optional last-seen time.

QUESTION 5 – SMART HOME SURVEILLANCE AND INTRUSION DETECTION

The fifth scenario uses a fixed entrance camera and requires significant-motion detection, discrimination between real movement and camera noise/background changes, tracking of moving objects, suspicious-activity detection, alerts for unauthorized entry into a predefined region and real-time operation on limited computational resources. The source specifically asks for background subtraction, filtering, morphology, contour/feature detection, motion analysis and tracking.

5.1 Proposed Pipeline

```
Fixed Camera
↓
Frame Capture
↓
ROI Selection
↓
Background Subtraction
↓
Blur / Noise Filtering
↓
Shadow / Illumination Handling
↓
Morphological Cleanup
↓
Contour Detection
↓
Object Filtering
↓
Object Tracking
↓
ROI Entry / Suspicious Motion Analysis
↓
Alert + Snapshot + Log
```

5.2 Background Subtraction

For a fixed camera, a background model can estimate the static scene and identify foreground pixels that differ significantly. OpenCV provides background-subtraction approaches suitable for surveillance. The model should adapt slowly enough to avoid treating a temporarily stationary intruder as background too quickly.

5.3 Filtering

Gaussian blur or median filtering can suppress small sensor noise before foreground extraction. Filtering should preserve the larger contours associated with people or objects.

5.4 Morphological Cleanup

- Opening removes small isolated foreground pixels.
- Closing fills small holes in detected silhouettes.
- Dilation can connect fragmented components.
- Erosion can remove tiny structures.
- Contour-area filtering rejects objects below a minimum size.

QUESTION 5 – FALSE ALARMS, TRACKING AND ALERTS

5.5 Shadows

Shadows can be incorrectly detected as moving foreground. Shadow-aware background subtraction can help identify pixels whose intensity changes while chromaticity remains relatively consistent. Additional contour and shape constraints can reduce shadow-generated detections.

5.6 Illumination Changes

Sudden changes in lighting can affect many pixels at once. The system can use adaptive background modelling, temporal smoothing and scene-change thresholds. A global illumination event should not automatically create an intrusion alert.

5.7 Trees and Dynamic Backgrounds

Trees, curtains and other moving background elements may generate persistent foreground regions. Restricting the region of interest, tuning background-learning rate and applying minimum object-size filters reduces these false positives.

5.8 Object Tracking

After contours or object detections are obtained, each object receives a track ID. The tracker stores centroid, bounding box, velocity estimate and last-seen time. Tracking prevents a single person from generating a separate alert for every frame.

5.9 Unauthorized Region Detection

A polygonal region can be defined around a doorway or restricted area. When a tracked object enters that polygon and the system determines that the event meets the configured significance criteria, an intrusion alert is generated.

```
if object_track.centroid_inside(restricted_region):
    if object_track.persistence >= minimum_frames:
        if not object_track.alerted:
            trigger_intrusion_alert(object_track)
            object_track.alerted = True
```

5.10 Alert Design

Alert Component	Purpose
Timestamp	When the event occurred.
Camera ID	Which camera generated the event.
Object track ID	Which tracked object triggered it.
Region	Where the object entered.
Snapshot	Visual evidence, subject to privacy policy.
Severity	Priority for the homeowner/operator.
Status	New, acknowledged or resolved.

6. COMPARATIVE ANALYSIS OF ALL FIVE SOLUTIONS

Although all five scenarios use OpenCV, the dominant computer-vision pipeline differs according to the constraint and required output.

Scenario	Core Vision Task	Primary Technique	Key Constraint
Face Recognition	Identity recognition	Face detection + embeddings + matching	Low false recognition + real time
Traffic Detection	Vehicle counting/violation	Detection + tracking + line crossing	Multiple objects + dynamic scene
Document Processing	Image enhancement/OCR preparation	Filtering + threshold + morphology + segmentation	High OCR accuracy + batch scale
Attendance	Identity + persistence	Face recognition + tracking + state	No duplicate attendance
Home Surveillance	Motion/intrusion	Background subtraction + contours + tracking	Low false alarms + low compute

6.1 Common Optimization Principles

- Restrict processing to regions of interest.
- Resize frames/images when full resolution is unnecessary.
- Use temporal information rather than making independent decisions for every frame.
- Use lightweight models on constrained hardware.
- Separate detection from tracking so expensive recognition is not repeated unnecessarily.
- Filter low-quality observations before expensive downstream processing.
- Log confidence and intermediate states to support evaluation.

6.2 Accuracy vs Speed Trade-Off

Higher-resolution input and more sophisticated models can improve detection or recognition but increase latency and resource usage. The correct operating point depends on the application. Face recognition needs sufficient facial detail, traffic counting needs stable object tracks, document processing prioritizes OCR quality over instantaneous response, while home surveillance prioritizes low false alarms and continuous operation.

6.3 Evaluation Metrics

Application	Useful Metrics
Face Recognition	Accuracy, precision, recall, false acceptance rate, false rejection rate, FPS.
Traffic	Detection precision/recall, counting error, ID switches, FPS.
Documents	OCR character accuracy, word accuracy, processing time/page.
Attendance	Recognition accuracy, duplicate rate, missed attendance, latency.
Surveillance	False alarms/hour, detection recall, alert latency, FPS.

7. IMPLEMENTATION GUIDELINES AND PSEUDOCODE

7.1 Common OpenCV Frame Loop

```
cap = cv2.VideoCapture(source)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = preprocess(frame)
    detections = detect(frame)

    tracks = tracker.update(detections)

    for track in tracks:
        result = analyze(track)
        visualize(frame, track, result)

    cv2.imshow("System", frame)

    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

7.2 Software Modules

Module	Responsibilities
capture.py	Camera/file acquisition and frame control.
preprocess.py	Resize, filtering, grayscale/normalization.
detector.py	Face, vehicle or foreground detection.
features.py	Feature/embedding extraction.
tracker.py	Track creation, update and removal.
rules.py	Counting, attendance or intrusion logic.
storage.py	Logs, attendance, event records.
visualization.py	Boxes, labels, counts and alerts.
config.py	Thresholds, ROI, source and runtime settings.

7.3 Configuration Parameters

- Input resolution and frame sampling rate.
- Detection confidence threshold.
- Recognition similarity threshold.
- Minimum contour/object area.
- Tracker maximum missing frames.
- Virtual line or restricted-region coordinates.
- Background-learning rate.
- Retry/alert cooldown period.

7.4 Testing Strategy

- Test with normal lighting and low-light conditions.
- Test motion blur and partial occlusion.
- Test multiple objects/faces simultaneously.
- Test temporary detection failure.
- Test camera noise and illumination changes.
- Measure false positives and false negatives.
- Measure FPS and processing latency on target hardware.
- Stress-test long-running streams for memory leaks.

8. ACCURACY, REAL-TIME PERFORMANCE AND CONSTRAINT HANDLING

8.1 Accuracy Engineering

Accuracy is improved by combining preprocessing, appropriate detection models, quality filtering, temporal consistency and conservative decision thresholds. A system should be evaluated on representative data rather than relying only on a few successful demonstrations.

8.2 Real-Time Engineering

Real-time performance means that processing latency remains low enough for the intended application. The system should measure actual frames per second and end-to-end delay. A pipeline that detects at 30 FPS but displays results with a long queue delay is not truly real time.

8.3 Low-Cost Hardware Strategy

- Reduce input dimensions before expensive operations.
- Use region-of-interest processing.
- Use detection less frequently and tracking more frequently.
- Choose lightweight OpenCV/DNN models where acceptable.
- Avoid unnecessary disk writes in the frame loop.
- Use asynchronous logging and alert delivery.
- Profile each stage to find the actual bottleneck.

8.4 Constraint Matrix

Constraint	Design Response
Limited CPU/GPU	Resize, frame skipping, lightweight models.
Low latency	Pipeline stages efficiently; track between detections.
False recognition	Confidence thresholds + temporal confirmation.
False counting	Track IDs + one-time line crossing.
OCR errors	Illumination correction + thresholding + deskew.
Duplicate attendance	Session/student uniqueness + persistent tracks.
False intrusion alerts	ROI + morphology + persistence + tracking.
Variable lighting	Adaptive preprocessing/background modelling.

8.5 Responsible Deployment

Face recognition and attendance systems involve personal data. A real deployment should define access control, retention periods, purpose limitation, audit logging and appropriate consent or institutional policy. The computer-vision pipeline should not be treated as infallible; uncertain cases should be handled conservatively.

9. CONCLUSION AND FINAL ANSWER SUMMARY

The five constraint-based scenarios demonstrate how the same OpenCV toolkit can support very different computer-vision systems. Real-time campus security requires face detection, feature extraction, identity matching and false-recognition controls. Traffic violation detection requires vehicle detection, multi-object tracking, line-crossing logic and robustness against shadows and dynamic backgrounds. Automated document processing focuses on image enhancement, thresholding, morphology, geometric correction and OCR preparation. Multi-person attendance combines recognition with persistent tracking and duplicate-prevention logic. Smart-home surveillance combines background subtraction, contour analysis, tracking and restricted-region rules to reduce false alarms.

In every scenario, the solution should be designed as a pipeline rather than as one isolated algorithm. Acquisition quality affects preprocessing; preprocessing affects detection; detection affects tracking; and tracking affects the final decision. Real-time optimization must therefore consider the entire pipeline. The most important engineering principle is to balance accuracy, latency, computational resources and false-alarm cost according to the application's constraints.

10. SHORT EXAM-READY SUMMARY

Question	Pipeline in One Line
Q1 Face Recognition	Capture → preprocess → detect face → align → extract features → match → threshold → identify/log.
Q2 Traffic	Capture → ROI/preprocess → detect vehicles → track → line crossing → count/violation → visualize.
Q3 Documents	Scan → grayscale → filter → illumination correction → threshold → morphology → deskew → segment → OCR.
Q4 Attendance	Capture → detect faces → features → recognize → track identities → confirm → mark once → report.
Q5 Surveillance	Capture → ROI → background subtraction → filter → morphology → contours → track → region rule → alert.

11. REFERENCES / SOURCE BASIS

[1] CO5 Assessment 1 – Constraint-Based Problem Solving, supplied assignment PDF. The five problem statements and their required OpenCV pipeline components are the primary source basis for this document. 穀filecite學turn14file0學L2-L35像

[2] OpenCV documentation and standard computer-vision concepts may be consulted when implementing the proposed pipelines. Specific model/library choices should be validated against the hardware and dataset used for the student's implementation.

Student Declaration

I K Nitheesh Kumar (192325090) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the Student: K Nitheesh Kumar